

UNIVERSIDAD DE LAS FUERZAS ARMADAS-ESPE
SEDE SANTO DOMINGO DE LOS TSÁCHILAS

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN

PERIODO : 202650 Marzo – Agosto 2026

ASIGNATURA : Fundamentos de Sistemas Web

TEMA : Taller: Construcción de interacciones web

INTEGRANTES : Sánchez Lánder

NIVEL-PARALELO - NRC: Cuarto

DOCENTE : Ing. Brito Casanova

FECHA DE ENTREGA : 17 de julio de 2026.

SANTO DOMINGO – ECUADOR

2026

Enlace del repositorio GitHub:

https://github.com/LanderSanchez1/Taller_P3_interacciones-web/tree/master

Tabla de Ilustraciones

Ilustración 1. Interfaz inicial del evaluador académico	13
Ilustración 2. Evaluación del estudiante caso1	13
Ilustración 3. Objeto JSON del estudiante caso 1	14
Ilustración 4. Evaluación del estudiante caso 2	15
Ilustración 5. Objeto JSON del estudiante caso 2	15
Ilustración 6. Evaluación de la estudiante caso 3	16
Ilustración 7. Objeto JSON de la estudiante caso 3	17
Ilustración 8. Evaluación del estudiante caso 4	17
Ilustración 9. Objeto JSON del estudiante caso 4	18
Ilustración 10. Evaluación de la estudiante caso 5	19
Ilustración 11. Objeto JSON de la estudiante caso 5	19
Ilustración 12. Estadísticas generales del curso	20
Ilustración 13. Ranking de estudiantes	20
Ilustración 14. Validación de nombre repetido	21

1. Descripción de la actividad.

En esta práctica se trabajó con los archivos base index.html y app.js para ampliar el funcionamiento de un evaluador académico desarrollado con JavaScript. Se realizaron modificaciones en la interfaz y en la lógica del programa para agregar una cuarta nota, permitir el ingreso de calificaciones decimales, actualizar las opciones de carrera y mostrar información adicional sobre el desempeño de cada estudiante.

Además, se implementaron nuevas funcionalidades como el cálculo del promedio especial sin la nota más baja, la identificación de la nota mayor y menor, la clasificación del rendimiento académico, la asignación de becas según la carrera y el promedio obtenido, y la generación de recomendaciones de estudio. También se creó un historial de estudiantes evaluados, con opciones para evitar nombres repetidos, buscar estudiantes, listar los registros en una tabla, obtener estadísticas generales del curso y mostrar un ranking de acuerdo con los promedios. Finalmente, toda la información de cada estudiante se organizó en un objeto JavaScript y se presentó en formato JSON.

2. Objetivo general.

Desarrollar y mejorar un evaluador académico utilizando HTML, Bootstrap y JavaScript, implementando nuevas funciones para calcular promedios, evaluar el rendimiento de los estudiantes y administrar un historial con información y estadísticas del curso.

3. Objetivos específicos.

- Implementar nuevas funciones en JavaScript para calcular promedios, identificar la nota mayor y menor, clasificar el rendimiento y asignar becas según las condiciones establecidas.
- Desarrollar un historial de estudiantes que permita registrar, buscar, listar y evitar registros duplicados, además de mostrar estadísticas generales del curso.

- Mejorar la interacción con el usuario mediante una interfaz que presente de forma clara los resultados, recomendaciones, tablas y la información del estudiante en formato JSON.

4. Marco conceptual

Modelo de Objetos del Documento (DOM)

El Document Object Model (DOM) es una interfaz que representa la estructura de un documento HTML como un árbol de objetos. Gracias al DOM, JavaScript puede acceder a los elementos de una página, modificar su contenido, cambiar estilos y actualizar la información mostrada al usuario sin necesidad de recargar el sitio web. Esta característica es la base para desarrollar aplicaciones web dinámicas e interactivas

Manejo de eventos en JavaScript

Los eventos permiten que una aplicación web responda a las acciones del usuario, como hacer clic en un botón o escribir información en un formulario. Para ello se utiliza el método `addEventListener()`, que asocia una función a un evento determinado y ejecuta el código correspondiente cuando este ocurre. Este mecanismo hace posible implementar interfaces más dinámicas y mejorar la experiencia del usuario (Flanagan, 2023).

Validación de datos en formularios

La validación de datos consiste en comprobar que la información ingresada por el usuario cumpla con los requisitos establecidos antes de ser procesada. En aplicaciones desarrolladas con JavaScript es común verificar que los campos no estén vacíos, que los valores numéricos se encuentren dentro de un rango permitido y que el formato de los datos sea correcto. Esto ayuda a reducir errores y mejora la calidad de la información recibida (Mozilla Contributors, 2025).

Arreglos y objetos en JavaScript

Los arreglos permiten almacenar colecciones de datos y realizar operaciones como recorrer, ordenar, buscar y calcular resultados mediante métodos integrados del lenguaje. Por su parte, los objetos permiten agrupar información relacionada mediante propiedades, facilitando la representación de entidades como un estudiante con sus notas, promedio y demás características. Estas estructuras son fundamentales para organizar la información dentro de una aplicación web (Freeman & Robson, 2024).

Formato JSON

JSON (JavaScript Object Notation) es un formato ligero para almacenar e intercambiar datos estructurados. Su sintaxis está basada en pares clave-valor y es ampliamente utilizada en aplicaciones web para representar información de manera organizada y facilitar el intercambio de datos entre diferentes sistemas. Debido a su simplicidad y compatibilidad con JavaScript, se ha convertido en uno de los estándares más utilizados en el desarrollo web moderno (Keiser & Lemire, 2023).

5. Desarrollo paso a paso.

Paso 1: Agregar la cuarta nota al formulario

Se modificó el archivo `index.html` para incluir un nuevo campo llamado Nota 4. Luego, en `app.js`, se actualizó la función que obtiene las notas para que lea cuatro valores y el promedio se calcule utilizando las cuatro calificaciones ingresadas.

Paso 2: Permitir notas con decimales

Los campos de notas se configuraron con `step="0.01"` para aceptar valores decimales. Además, el promedio final se mostró con dos decimales utilizando el método `toFixed(2)` para presentar un resultado más preciso.

Paso 3: Ajustar las carreras del formulario

Se modificó el selector de carrera para trabajar únicamente con las opciones requeridas: Tecnologías de la Información, Software y Sistemas. Esto permitió que la lógica del programa coincidiera con las carreras definidas en la actividad.

Paso 4: Crear el mensaje de beca académica

Se agregó un nuevo div en el HTML para mostrar el resultado de la beca. En JavaScript se creó la función `calcularBeca()`, la cual analiza la carrera y el promedio del estudiante para determinar si recibe una beca del 100 %, 80 %, 60 % o si no aplica.

Paso 5: Mostrar colores según el tipo de beca

El mensaje de beca se personalizó usando clases de Bootstrap. Las becas del 100 % se muestran en verde, las del 80 % en azul, las del 60 % en amarillo y el mensaje “Sin beca” en gris, facilitando la identificación visual del resultado.

Paso 6: Calcular el promedio especial sin la nota más baja

Se implementó la función `promedioSinMenor()`, que identifica la nota más baja del arreglo de notas, la elimina temporalmente y calcula el promedio únicamente con las notas restantes.

Paso 7: Mostrar la nota más alta y la más baja

Se crearon las funciones `notaMayor()` y `notaMenor()` para obtener la calificación máxima y mínima del estudiante. Estos valores se muestran en pantalla junto con el promedio especial.

Paso 8: Contar notas aprobadas, en recuperación y reprobadas

Mediante la función `contarEstadosNotas()` se recorrieron las notas del estudiante y se contó cuántas estaban aprobadas, cuántas en recuperación y cuántas reprobadas, generando un resumen del desempeño en las evaluaciones.

Paso 9: Generar una recomendación académica

Se desarrolló la función `recomendacionPorPromedio()`, que entrega una recomendación diferente según el rango del promedio obtenido, orientando al estudiante sobre acciones para mejorar o mantener su rendimiento.

Paso 10: Clasificar el rendimiento académico

Con la función `clasificarRendimiento()` el sistema clasifica el desempeño como Alto, Medio, Básico o Bajo, proporcionando una evaluación adicional del rendimiento general.

Paso 11: Crear el historial de estudiantes evaluados

Se creó un arreglo global llamado historial donde se almacenan los objetos de cada estudiante evaluado correctamente, permitiendo conservar la información durante la sesión.

Paso 12: Validar nombres repetidos

Antes de guardar un estudiante en el historial, el sistema verifica si ya existe otro registro con el mismo nombre (sin importar mayúsculas o minúsculas). Si existe, se muestra un mensaje de advertencia y no se agrega el duplicado.

Paso 13: Crear el botón para listar estudiantes

Se añadió el botón “Listar estudiantes”, el cual genera dinámicamente una tabla con todos los estudiantes registrados y muestra información como promedio, estado, rendimiento y beca.

Paso 14: Mostrar cuántos estudiantes han sido evaluados

Se implementó una tarjeta de estadísticas del curso que indica el total de estudiantes almacenados en el historial.

Paso 15: Contar aprobados, recuperación y reprobados del curso

Recorriendo el historial completo, el programa calcula cuántos estudiantes están aprobados, en recuperación y reprobados, mostrando un resumen general del curso.

Paso 16: Calcular el promedio general del curso

Se sumaron los promedios de todos los estudiantes del historial y se dividieron para el número total de registros, obteniendo el promedio general del curso.

Paso 17: Buscar estudiante por nombre

Se agregó un campo de búsqueda y un botón que permiten localizar un estudiante dentro del historial y mostrar nuevamente toda su información en pantalla.

Paso 18: Crear el ranking de estudiantes

Los estudiantes del historial se ordenaron de mayor a menor promedio utilizando sort(), generando un ranking académico visible en la tabla.

Paso 19: Mostrar el estudiante con mayor y menor promedio

A partir del ranking se identificó quién tiene el mejor promedio y quién posee el promedio más bajo, mostrando ambos resultados en la sección de estadísticas.

Paso 20: Mejorar el objeto final del estudiante

Finalmente, el objeto estudiante se amplió para incluir nombre, carrera, notas, promedio, promedio especial, nota mayor, nota menor, estado, rendimiento, beca y recomendación. Este objeto se presenta también en formato JSON para visualizar toda la información estructurada.

6. Código relevante explicado.

1. Obtención de las notas y cálculo del promedio

En este fragmento se obtiene el valor de las cuatro notas ingresadas por el usuario y se convierten a valores numéricos mediante parseFloat(). Posteriormente, se calcula el promedio general del estudiante utilizando el arreglo de notas, lo que permite trabajar con una cantidad fija de calificaciones y simplifica los cálculos posteriores.

```
const getNotas = () => [  
  parseFloat(document.getElementById('nota1').value) || 0,  
  parseFloat(document.getElementById('nota2').value) || 0,  
  parseFloat(document.getElementById('nota3').value) || 0,  
  parseFloat(document.getElementById('nota4').value) || 0,  
];  
  
const promedio = notas => notas.reduce((a,b)=>a+b,0)/notas.length;
```

2. Cálculo del promedio especial

Para obtener un promedio especial, el programa identifica la nota más baja del estudiante, la elimina temporalmente del arreglo y realiza nuevamente el cálculo del promedio con las notas restantes. Esta función permite obtener una evaluación alternativa del rendimiento académico.

```
const promedioSinMenor = notas => {  
  if (notas.length <=1) return promedio(notas);  
  const min = notaMenor(notas);  
  const restantes = notas.slice();  
  const idx = restantes.indexOf(min);  
  
  if (idx !== -1)  
    restantes.splice(idx,1);  
  
  return promedio(restantes);  
};
```

3. Clasificación del rendimiento y asignación de becas

Las siguientes funciones analizan el promedio obtenido por el estudiante para clasificar su rendimiento y determinar si cumple los requisitos para recibir una beca según la carrera seleccionada. Además, generan una recomendación académica personalizada.

```
const clasificarRendimiento = prom => {  
  if (prom >= 18) return 'Alto';  
  if (prom >= 14) return 'Medio';  
  if (prom >= 10) return 'Básico';  
  return 'Bajo';  
};
```

```
const calcularBeca = (carrera, prom) => {  
  if (!carrera)  
    return {porcentaje:0, clase:'secondary', texto:'Sin beca'};
```

```

if (carrera === 'TI' && prom > 18)

    return {porcentaje:100, clase:'success', texto:'Beca 100%'};

if (carrera === 'Software' && prom > 17)

    return {porcentaje:80, clase:'primary', texto:'Beca 80%'};

if (carrera === 'Sistemas' && prom > 16)

    return {porcentaje:60, clase:'warning', texto:'Beca 60%'};

return {porcentaje:0, clase:'secondary', texto:'Sin beca'};

};

```

4. Validación y creación del objeto estudiante

Esta función verifica que el usuario haya ingresado correctamente el nombre, la carrera y las notas. Después de realizar todos los cálculos, crea un objeto que reúne toda la información del estudiante, facilitando su almacenamiento y reutilización en el resto del programa.

```

return {
    nombre,
    carrera,
    notas,
    promedio: Number(prom.toFixed(2)),
    promedioEspecial: Number(promEspecial.toFixed(2)),
    notaMayor: mayor,
    notaMenor: menor,
    estadosNotas,
    estado,
    rendimiento,
    beca,

```

```
recomendacion  
};
```

5. Registro del historial de estudiantes

Para conservar la información de todos los estudiantes evaluados, se utiliza un arreglo global llamado historial. Antes de agregar un nuevo registro, el programa verifica que no exista otro estudiante con el mismo nombre, evitando así duplicados.

```
const historial = [];  
  
const agregarHistorial = (estudiante) => {  
  const existe = historial.some(  
    e => e.nombre.toLowerCase() === estudiante.nombre.toLowerCase()  
  );  
  
  if (existe) {  
    resultadoEl.className = 'alert mt-4 alert-warning';  
    resultadoEl.innerText = 'Ya existe un estudiante con ese nombre en el historial.';  
    return false;  
  }  
  
  historial.push(estudiante);  
  return true;  
};
```

6. Generación de estadísticas del curso

Una vez registrados varios estudiantes, el sistema recorre el historial para calcular el total de estudiantes, la cantidad de aprobados, estudiantes en recuperación y reprobados, además del promedio general del curso y el ranking académico.

```
historial.forEach(e => {  
  if (e.promedio >= UMBRAL_APROBADO)
```

```

        aprob++;
    else if (e.promedio >= UMBRAL_RECUP)
        rec++;
    else
        rep++;

    sumaProm += e.promedio;
});

const promGeneral = sumaProm / historial.length;

```

7. Eventos de los botones

La interacción entre el usuario y la aplicación se realiza mediante eventos. Cada botón ejecuta una función específica: evaluar al estudiante, limpiar el formulario, listar los registros almacenados y buscar un estudiante por su nombre.

```

document.getElementById('btnEvaluar').addEventListener('click', function(){
    limpiarUI();
    const obj = validarYCrearObjeto();

    if (!obj) return;

    renderResultado(obj);

    const agregado = agregarHistorial(obj);

    if (agregado)
        renderCursoInfo();
});

```

7. Capturas de pantalla con rótulo y título.

Interfaz inicial del evaluador académico

Evaluador académico

Ingrese los datos y calcule el estado del estudiante.

Nombre

Ejemplo: Ana Torres

Carrera

Selecione...

Nota 1

Nota 2

Nota 3

Nota 4

Evaluar

Limpiar

Listar estudiantes

Buscar estudiante

Nombre a buscar

Buscar

En esta imagen se muestra la pantalla principal de la aplicación al iniciar su ejecución. Se puede observar el formulario con los campos para ingresar el nombre del estudiante, la carrera y las cuatro notas, además de los botones para evaluar, limpiar, listar estudiantes y realizar búsquedas. En este estado aún no se ha ingresado información, por lo que no se muestran resultados ni estadísticas.

En esta prueba se ingresaron los datos de la estudiante Ana Torres, perteneciente a la carrera de Tecnologías de la Información. El sistema calculó correctamente el promedio, determinó que el estado académico era aprobado, asignó la beca del 100 % y mostró información adicional como la nota mayor, la nota menor, el promedio especial, la recomendación académica y el nivel de rendimiento.

Ilustración 2. Evaluación del estudiante caso1

En esta imagen se observa el objeto generado en formato JSON, donde se almacenan todos los datos del estudiante junto con los resultados calculados. Esta estructura facilita la organización y el manejo de la información dentro de la aplicación.

```
{
  "nombre": "Ana Torres",
  "carrera": "TI",
  "notas": [
    19,
    18.5,
    20,
    19
  ],
  "promedio": 19.13,
  "promedioEspecial": 19.33,
  "notaMayor": 20,
  "notaMenor": 18.5,
  "estadosNotas": {
    "aprob": 4,
    "rec": 0,
    "rep": 0
  },
  "estado": "Aprobado",
  "rendimiento": "Alto",
  "beca": {
    "porcentaje": 100,
    "clase": "success",
    "texto": "Beca 100%"
  },
  "recomendacion": "Mantener el desempeño y apoyar a compañeros."
}
```

Ilustración 3. Objeto JSON del estudiante caso 1

Evaluación del estudiante Carlos Pérez

En esta prueba se registró al estudiante Carlos Pérez de la carrera de Software. El sistema calculó el promedio obtenido y asignó correctamente la beca del 80 %, además de mostrar el estado académico, las estadísticas de notas y la recomendación correspondiente.

Buscar estudiante

Carlos Pérez

Buscar

El estudiante Carlos Pérez está: Aprobado. Promedio: 17.63

Beca 80% (80%)

Notas: 17, 18, 17.5, 18 | Mayor: 18 | Menor: 17 | Prom. esp.: 17.83

Recomendación: Reforzar temas específicos.

Rendimiento: Medio

Ilustración 4. Evaluación del estudiante caso 2

Objeto JSON del estudiante Carlos Pérez

Se presenta el objeto JSON generado para Carlos Pérez, el cual contiene toda la información ingresada y los datos calculados automáticamente por el programa.

```
{
  "nombre": "Carlos Pérez",
  "carrera": "Software",
  "notas": [
    17,
    18,
    17.5,
    18
  ],
  "promedio": 17.63,
  "promedioEspecial": 17.83,
  "notaMayor": 18,
  "notaMenor": 17,
  "estadosNotas": {
    "aprob": 4,
    "rec": 0,
    "rep": 0
  },
  "estado": "Aprobado",
  "rendimiento": "Medio",
  "beca": {
    "porcentaje": 80,
    "clase": "primary",
    "texto": "Beca 80%"
  },
  "recomendacion": "Reforzar temas específicos."
}
```

Ilustración 5. Objeto JSON del estudiante caso 2

Evaluación de la estudiante María López

En este caso se evaluó a la estudiante María López de la carrera de Sistemas. La aplicación verificó el promedio obtenido, asignó la beca del 60 % y presentó correctamente toda la información relacionada con su rendimiento académico.

Buscar estudiante

María López

Buscar

El estudiante María López está: Aprobado. Promedio: 16.38

Beca 60% (60%)

Notas: 16, 17, 16.5, 16 | Mayor: 17 | Menor: 16 | Prom. esp.: 16.50

Recomendación: Reforzar temas específicos.

Rendimiento: Medio

Ilustración 6. Evaluación de la estudiante caso 3

Objeto JSON de la estudiante María López

La imagen muestra la representación en formato JSON de la información de María López, permitiendo visualizar de forma estructurada todos los datos almacenados por el sistema.


```

{
  "nombre": "María López",
  "carrera": "Sistemas",
  "notas": [
    16,
    17,
    16.5,
    16
  ],
  "promedio": 16.38,
  "promedioEspecial": 16.5,
  "notaMayor": 17,
  "notaMenor": 16,
  "estadosNotas": {
    "aprob": 4,
    "rec": 0,
    "rep": 0
  },
  "estado": "Aprobado",
  "rendimiento": "Medio",
  "beca": {
    "porcentaje": 60,
    "clase": "warning",
    "texto": "Beca 60%"
  },
  "recomendacion": "Reforzar temas específicos."
}

```

Ilustración 7. Objeto JSON de la estudiante caso 3

Evaluación del estudiante José Ramírez

En esta prueba se ingresaron las calificaciones de José Ramírez. El sistema determinó que el estudiante se encontraba en estado de recuperación, mostró el promedio correspondiente y presentó la recomendación académica para mejorar su desempeño.

Buscar estudiante

José Ramírez

Buscar

El estudiante José Ramírez está: Recuperación. Promedio: 11.50

Sin beca

Notas: 12, 11, 10, 13 | Mayor: 13 | Menor: 10 | Prom. esp.: 12.00

Recomendación: Asistir a tutorías y practicar ejercicios.

Rendimiento: Básico

Ilustración 8. Evaluación del estudiante caso 4

Objeto JSON del estudiante José Ramírez

En esta captura se observa el objeto JSON generado para José Ramírez, incluyendo todas las propiedades calculadas como promedio, rendimiento, estado y recomendación.

```
{
  "nombre": "José Ramírez",
  "carrera": "TI",
  "notas": [
    12,
    11,
    10,
    13
  ],
  "promedio": 11.5,
  "promedioEspecial": 12,
  "notaMayor": 13,
  "notaMenor": 10,
  "estadosNotas": {
    "aprob": 0,
    "rec": 4,
    "rep": 0
  },
  "estado": "Recuperación",
  "rendimiento": "Básico",
  "beca": {
    "porcentaje": 0,
    "clase": "secondary",
    "texto": "Sin beca"
  },
  "recomendacion": "Asistir a tutorías y practicar ejercicios."
}
```

Ilustración 9. Objeto JSON del estudiante caso 4

Evaluación de la estudiante Sofía Mendoza

En este caso se registró a la estudiante Sofía Mendoza con calificaciones bajas. El sistema la clasificó correctamente como reprobada, indicó que no aplica para una beca y mostró la información correspondiente sobre su rendimiento académico.

Buscar estudiante

Sofía Mendoza

Buscar

El estudiante Sofía Mendoza está: Reprobado. Promedio: 7.63

Sin beca

Notas: 7, 8, 9, 6.5 | Mayor: 9 | Menor: 6.5 | Prom. esp.: 8.00

Recomendación: Repetir contenidos base y solicitar acompañamiento.

Rendimiento: Bajo

Ilustración 10. Evaluación de la estudiante caso 5

Objeto JSON de la estudiante Sofía Mendoza

La imagen presenta el objeto JSON correspondiente a Sofía Mendoza, donde se almacenan todos los datos del estudiante junto con los resultados obtenidos durante la evaluación.

```
{
  "nombre": "Sofía Mendoza",
  "carrera": "Software",
  "notas": [
    7,
    8,
    9,
    6.5
  ],
  "promedio": 7.63,
  "promedioEspecial": 8,
  "notaMayor": 9,
  "notaMenor": 6.5,
  "estadosNotas": {
    "aprob": 0,
    "rec": 0,
    "rep": 4
  },
  "estado": "Reprobado",
  "rendimiento": "Bajo",
  "beca": {
    "porcentaje": 0,
    "clase": "secondary",
    "texto": "Sin beca"
  },
  "recomendacion": "Repetir contenidos base y solicitar acompañamiento."
}
```

Ilustración 11. Objeto JSON de la estudiante caso 5

Estadísticas generales del curso

En esta captura se muestran las estadísticas generadas automáticamente por el sistema después de registrar varios estudiantes. Se presenta el número total de estudiantes evaluados, la cantidad de aprobados, estudiantes en recuperación y reprobados, además del promedio general del curso y la información del mejor y peor promedio.

Estadísticas del Curso

Total estudiantes: 5

Aprobados: 3 | Recuperación: 1 | Reprobados: 1

Promedio general del curso: 14.45 | Top: Ana Torres (19.13)

Mejor promedio: Ana Torres (19.13) — Peor promedio: Sofía Mendoza (7.63)

Ilustración 12. Estadísticas generales del curso

Ranking de estudiantes

En esta imagen se observa la tabla con el historial de estudiantes registrados, ordenados según su promedio académico. Además de mostrar el ranking, la tabla presenta información como la carrera, las notas, el promedio especial, el estado académico, el rendimiento y la beca obtenida por cada estudiante.

Ranking de Estudiantes (Ordenados por Promedio Alto)

Nombre	Carrera	Notas	Promedio	Promedio esp.	Mayor	Menor	Estado	Rendimiento	Beca
Ana Torres	TI	19, 18.5, 20, 19	19.13	19.33	20	18.5	Aprobado	Alto	Beca 100%
Carlos Pérez	Software	17, 18, 17.5, 18	17.63	17.83	18	17	Aprobado	Medio	Beca 80%
María López	Sistemas	16, 17, 16.5, 16	16.38	16.50	17	16	Aprobado	Medio	Beca 60%
José Ramírez	TI	12, 11, 10, 13	11.50	12.00	13	10	Recuperación	Básico	Sin beca
Sofía Mendoza	Software	7, 8, 9, 6.5	7.63	8.00	9	6.5	Reprobado	Bajo	Sin beca

Ilustración 13. Ranking de estudiantes

Validación de nombre repetido

En esta prueba se intentó registrar nuevamente a la estudiante Ana Torres. El sistema detectó que el nombre ya existía en el historial y mostró un mensaje de advertencia, evitando la creación de un registro duplicado y garantizando la integridad de la información almacenada.

Evaluador académico

Ingrese los datos y calcule el estado del estudiante.

Nombre		Carrera	
Ana Torres		Software	
Nota 1	Nota 2	Nota 3	Nota 4
7	8	9	6.5
Evaluar	Limpiar	Listar estudiantes	

Buscar estudiante

Sofía Mendoza	Buscar
--	---------------------------------------

Ya existe un estudiante con ese nombre en el historial.

Ilustración 14. Validación de nombre repetido

8. Pruebas realizadas.

Para comprobar el funcionamiento de la aplicación se realizaron varias pruebas utilizando diferentes estudiantes y combinaciones de notas. En el primer caso se registró a Ana Torres con la carrera de Tecnologías de la Información y un promedio superior a 18, verificando que el sistema la clasificara como aprobada, con rendimiento alto y beca del 100 %. En el segundo caso se evaluó a Carlos Pérez de Software, comprobando la asignación correcta de la beca del 80 %. De igual manera, con María López de Sistemas se verificó la entrega de la beca del 60 %.

También se probaron escenarios con promedios más bajos. El estudiante José Ramírez obtuvo un promedio dentro del rango de recuperación, por lo que el sistema mostró el estado correspondiente, un rendimiento básico y la recomendación académica adecuada. Finalmente, con Sofía Mendoza se ingresaron notas inferiores al promedio mínimo requerido, comprobando que la aplicación la clasificara como reprobada y sin derecho a beca.

Además de las pruebas anteriores, se intentó registrar nuevamente a Ana Torres para verificar la validación de nombres repetidos. En este caso, el sistema mostró un mensaje de advertencia indicando que el estudiante ya existía en el historial y evitó agregar un registro duplicado. Finalmente, se comprobó el funcionamiento del listado de estudiantes, la búsqueda por nombre, las estadísticas generales del curso, el promedio general y el ranking, confirmando que todas las funcionalidades implementadas operan correctamente.

9. Análisis de resultados.

Durante el desarrollo de la práctica se comprobó que todas las funcionalidades implementadas trabajan de manera integrada. El sistema permite ingresar los datos del estudiante, validar la información, calcular el promedio utilizando cuatro notas y determinar automáticamente el estado académico correspondiente. Además, los resultados se muestran de forma clara en la interfaz, facilitando la interpretación de la información.

Las funciones adicionales, como el cálculo del promedio especial, la identificación de la nota mayor y menor, la asignación de becas y la generación de recomendaciones, aportan información más completa sobre el rendimiento de cada estudiante. Esto permitió verificar que JavaScript puede utilizarse para realizar diferentes procesos lógicos y actualizar dinámicamente el contenido de una página web sin necesidad de recargarla.

Por otra parte, el historial de estudiantes demostró el uso práctico de arreglos y objetos para almacenar información, mientras que las funciones de búsqueda, listado, ranking y estadísticas generales permitieron analizar los datos registrados de manera organizada. En conjunto, los resultados obtenidos muestran que la aplicación cumple con los objetivos planteados y ofrece una interfaz funcional para gestionar y evaluar la información académica de varios estudiantes.

10. Conclusiones.

- Durante la práctica fue posible comprender cómo JavaScript permite agregar interactividad a una página web, procesando la información ingresada por el usuario y mostrando los resultados de forma inmediata.
- La implementación del historial de estudiantes y las estadísticas del curso permitió entender mejor el uso de arreglos, objetos y funciones para organizar y manejar datos dentro de una aplicación.
- El desarrollo de nuevas funcionalidades, como el cálculo de becas, el promedio especial y el ranking de estudiantes, ayudó a reforzar la lógica de programación y el manejo del DOM para actualizar la interfaz de manera dinámica.

11. Recomendaciones.

- Planificar la estructura del programa antes de empezar a programar, ya que esto facilita la organización del código y hace más sencillo agregar nuevas funcionalidades.
- Validar siempre los datos ingresados por el usuario para evitar errores y asegurar que los resultados obtenidos sean correctos.

- Probar la aplicación con diferentes casos y valores para verificar que todas las funciones trabajen correctamente y detectar posibles errores antes de finalizar el desarrollo.

12. Criterio personal.

En este taller aprendí a desarrollar una aplicación web más completa utilizando JavaScript, combinando la lógica de programación con la interacción de los elementos del HTML. Comprendí mejor cómo usar funciones, objetos, arreglos y eventos para procesar información y mostrar resultados en la página sin necesidad de recargarla.

La parte que más me costó fue organizar todas las funciones para que trabajaran correctamente entre sí, especialmente al momento de crear el historial de estudiantes y actualizar las estadísticas del curso. Sin embargo, al ir probando cada función por separado fue más fácil encontrar y corregir los errores.

La funcionalidad que considero más útil es el historial de estudiantes, ya que permite almacenar la información, realizar búsquedas, generar un ranking y obtener estadísticas generales del curso. Este taller me ayudó a comprender cómo se construyen aplicaciones web interactivas con JavaScript y cómo se puede manipular el DOM para ofrecer una mejor experiencia al usuario.

13. Bibliografía complementaria.

Flanagan, D. (2023). JavaScript: The Definitive Guide (8th ed.). O'Reilly Media.

Freeman, E., & Robson, E. (2024). Head First JavaScript Programming (2nd ed.). O'Reilly Media.

Keiser, J., & Lemire, D. (2023). On-Demand JSON: A Better Way to Parse Documents? arXiv. <https://doi.org/10.48550/arXiv.2312.17149> (arXiv)

Mozilla Contributors. (2025). Client-side form validation. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms/Form_validation (MDN Web Docs)

Mozilla Contributors. (2025). Web forms. MDN Web Docs.
https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Forms
(MDN Web Docs)

Mozilla Contributors. (2025). Working with JSON. MDN Web Docs.
https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Scripting/JSON (MDN Web Docs)